西南交通大学
Southwest Jiaotong University

多处理器，多核

Multiprocessor, Multicore and Real-Time Scheduling

与实时调度



Xiangyi Chen

xiangyichen@swjtu.edu.cn

Outline

- **Multiprocessor and Multicore Scheduling** 多处理器和多核调度
  - Granularity 细粒度
  - Design Issues 设计问题
  - Process Scheduling 进程调度
  - Thread Scheduling 线程调度
  - Multicore Thread Scheduling 多核线程调度
- **Real-Time Scheduling** 实时调度
  - Background of Real-Time Process
  - Characteristics of Real-Time OS
  - Real-Time Scheduling
  - Deadline Scheduling 截止时间调度
  - Rate Monotonic Scheduling → 速率单调调度
  - Priority Inversion 优先级反转

Objectives

- Understand the concept of thread granularity
- Key design issues in multiprocessor thread scheduling and some of the key approaches to scheduling
- Understand the requirements imposed by real-time scheduling

西南交通大学
Southwest Jiaotong University

1.1 Classifications of Multiprocessor Systems

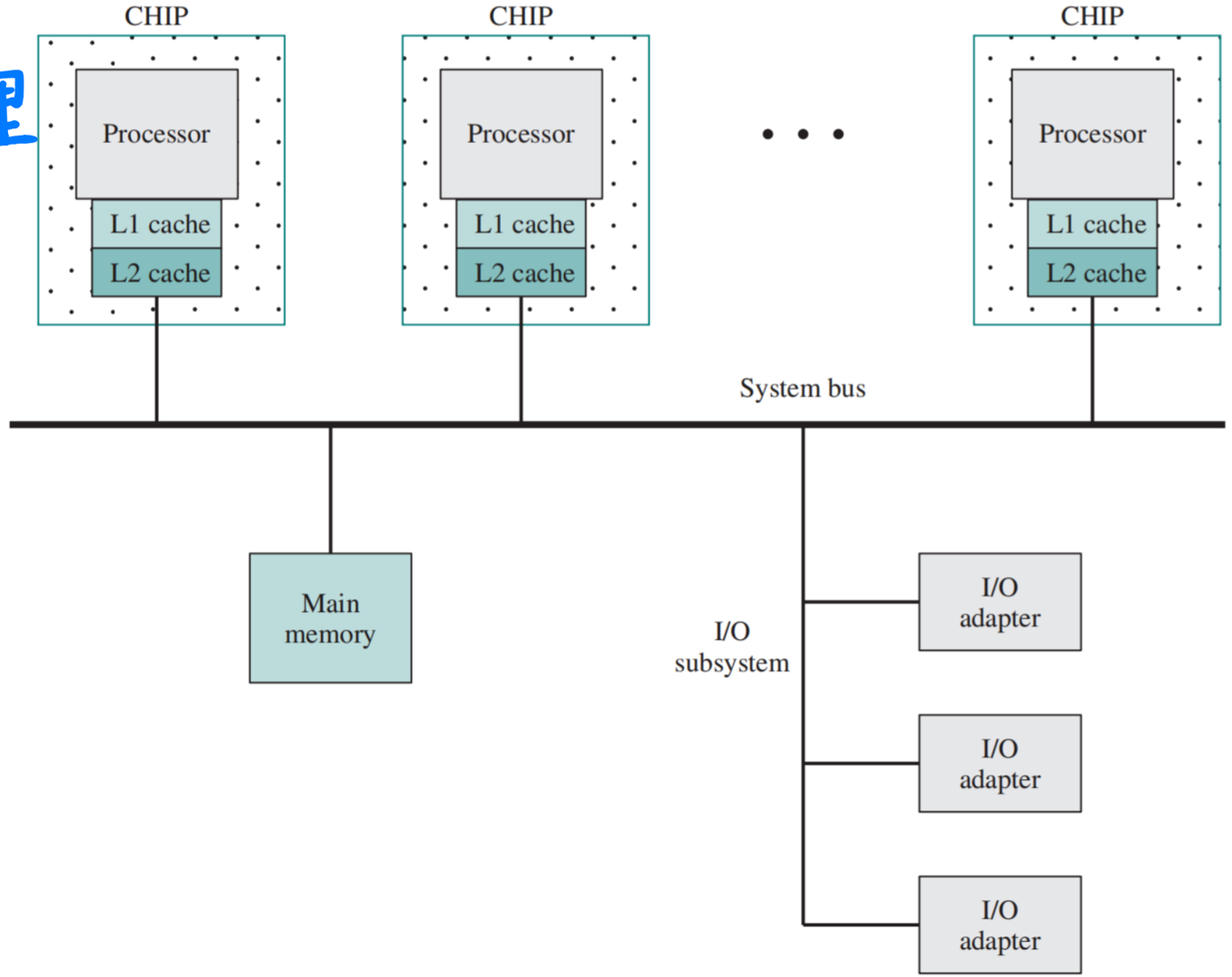
西南交通大学 Southwest Jiaotong University

- **Loosely coupled or distributed multiprocessor, or cluster**
  - consists of a collection of relatively autonomous systems, each processor having its own main memory and I/O channels
- **Functionally specialized processors**
  - there is a master processor with general-purpose
  - specialized processors are controlled by the master processor and provide services to it, such as I/O processor
- **Tightly coupled multiprocessor**
  - consists of a set of processors that share a common main memory and are under the integrated control of an operating system

Symmetric Multiprocessor Organization

对称多处理器结构

并行



Synchronization Granularity among Processes

进程间同步粒度

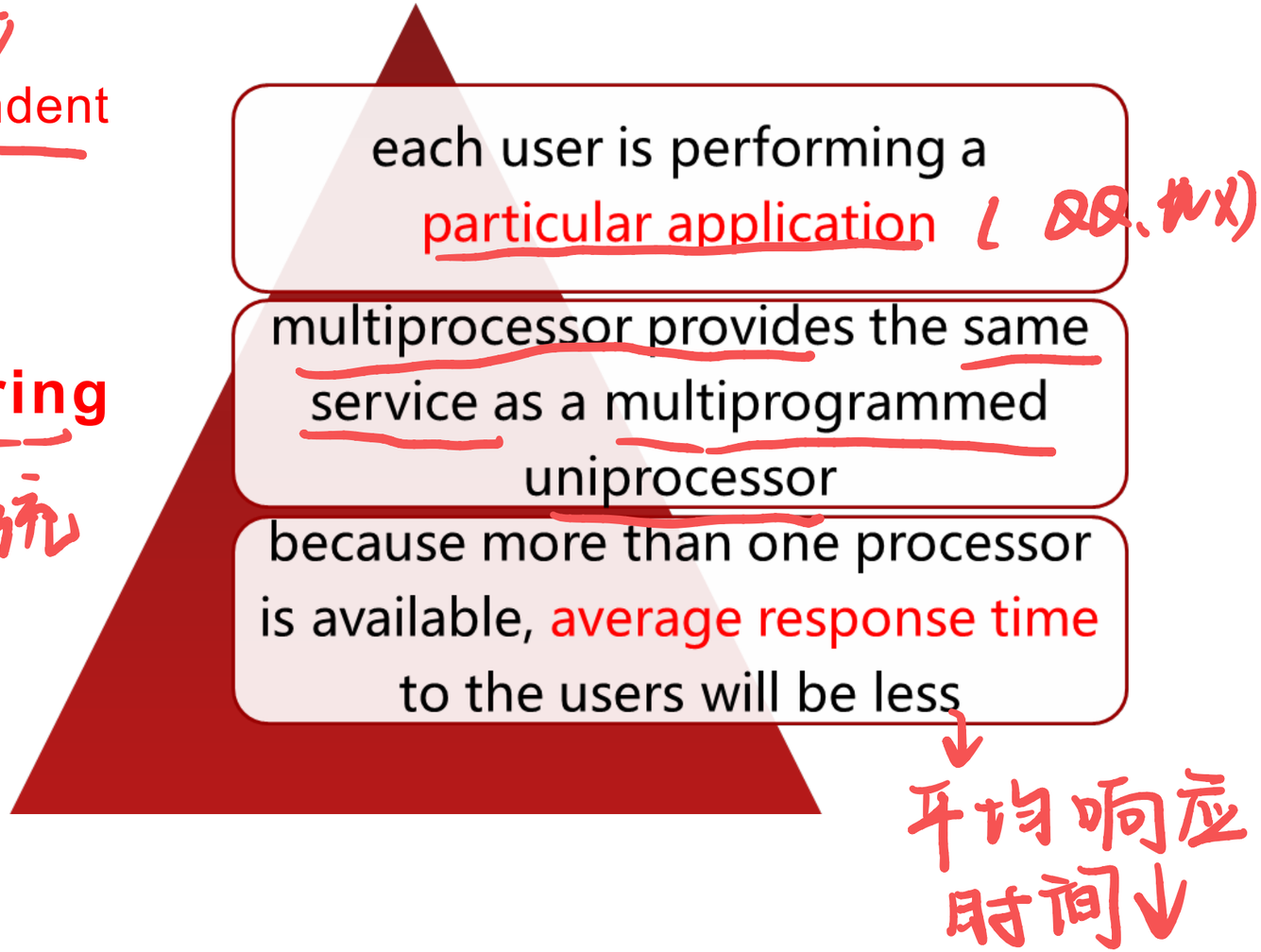
Grain Size	Description	Synchronization Interval (Instructions)
Fine 细粒度 Instruction-Level 指令级	Parallelism inherent in a single instruction stream. 单指令流固有的并行性	<20 指令数 (硬件自动实现指令并行)
Medium 中粒度 Thread-Level 线程级	Parallel processing or multitasking within a single application 单个应用内并行多任务	20-200
Coarse 粗粒度 Process-Level 进程级	Multiprocessing of concurrent processes in a multiprogramming environment	200-2000
Very Coarse 级粗粒度 Processor-Level 处理器级	Distributed processing across network nodes to form a single computing environment 跨网络节点的分布式处理	2000-1M
Independent Process-Level 进程级	Multiple unrelated processes 无关进程	not applicable

Independent Parallelism（粗）

西南交通大学 Southwest Jiaotong University

- **No explicit synchronization among processes** 进程间无显式同步
  - each represents a separate, independent application

- **Typical use is in a time-sharing system** 分时系统



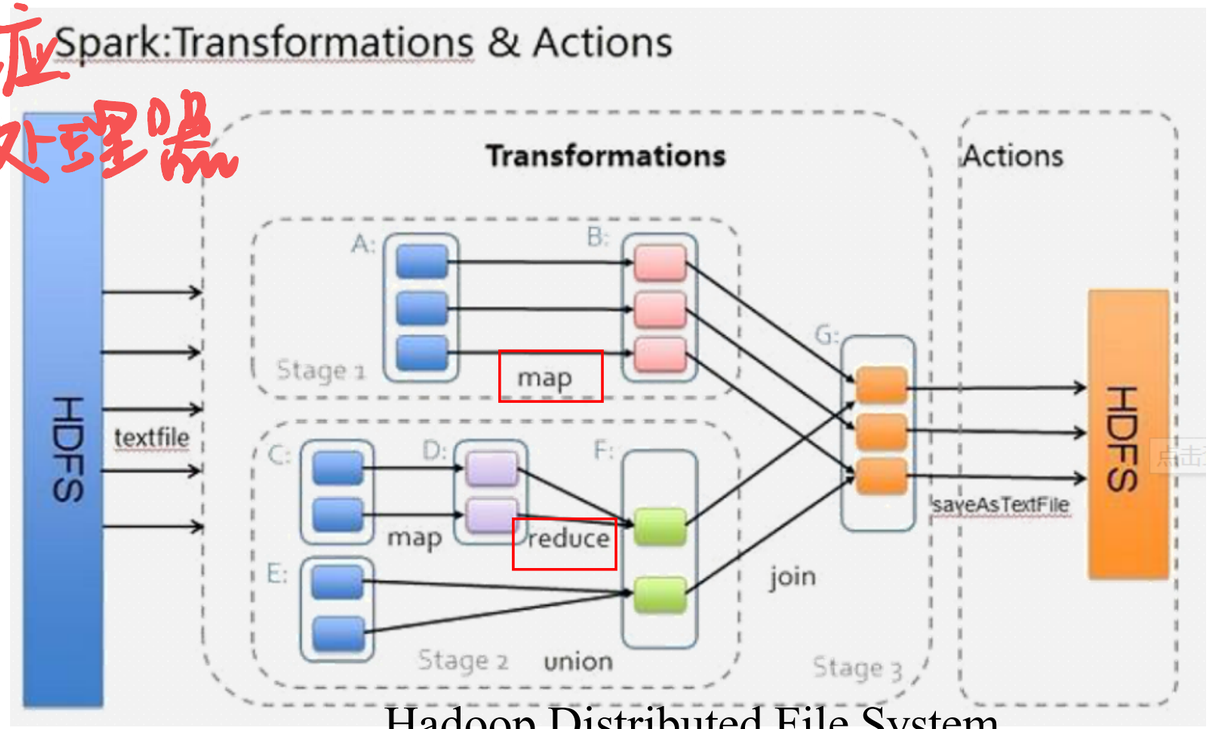
Coarse and Very Coarse-Grained Parallelism

西南交通大学 Southwest Jiaotong University

粗粒度 极粗粒度 进程间存在同步，但同步粒度粗

- **Synchronization among processes, but at a very gross level**
- **Good for concurrent processes running on a multiprogrammed uniprocessor** 适用于在多道程序设计的单处理器系统→单核CPU分时多进程
  - can be supported on a multiprocessor with little or no change to user software 也适应多处理器





Hadoop Distributed File System

Medium-Grained Parallelism

中粒度=线程并行（多核）

- **Single application can be effectively implemented as a collection of threads within a single process** → 显式指定
  - programmer must explicitly specify the potential parallelism of an application
  - there needs to be a high degree of coordination and interaction among the threads of an application, leading to a medium-grain level of synchronization 同步性更高
- **Because the various threads of an application interact so frequently, scheduling decisions concerning one thread may affect the performance of the entire application**

一个影响全布

Fine-Grained Parallelism

细粒度平行<硬件>

- Represents a much more complex use of parallelism than is found in the use of threads
- Is a specialized and fragmented area with many different approaches

专业且碎片化

Design Issues

调度方案 ——① 应用的并行粒度
② 可用处理器(核)数

- **The approach is depend on:**
  - the degree of granularity of applications
  - the number of processors available

- **Scheduling on a multiprocessor involves three interrelated issues:**
  - assignment of processes to processors (哪个进程)
  - use of multiprogramming on individual processors (是否分时多进程)
  - actual dispatching of a process (真实调度算法(RR, FCFS))

西南交通大学 Southwest Jiaotong University

Assignment of Processes to Processors

西南交通大学 Southwest Jiaotong University

进程在处理器上如何分配

- **Assuming all processors are equal, treat processors as a pooled resource and assign processes to processors on demand**
  - static or dynamic needs to be determined **(Assignment Policy)**

共享资源池　静态/动态

- **Static Assignment**
  - a process is permanently assigned to one processor from activation and its completion.
  - A Short-Term queue is maintained for each processor
  - **advantage** is that there may be less overhead in the scheduling function
  - **disadvantage** is unbalance load among different processor
  - **Solution**: common queue can be used to as global queue for all processes

调度函数开销更低

不同处理器负载不均衡

使用全局公共队列

Assignment of Processes to Processors

西南交通大学 Southwest Jiaotong University

动态负载均衡分均

- **Dynamic Load Balancing Assignment**
  - Threads are moved from a queue for one processor to a queue for another processor (Linux)

- **Both dynamic and static methods require some way of assigning a process to a processor (OS architecture)**
  - Master/Slave
  - Peer (对等)

主从式 动态/静态分配 由底层 OS 分配
对等式 是

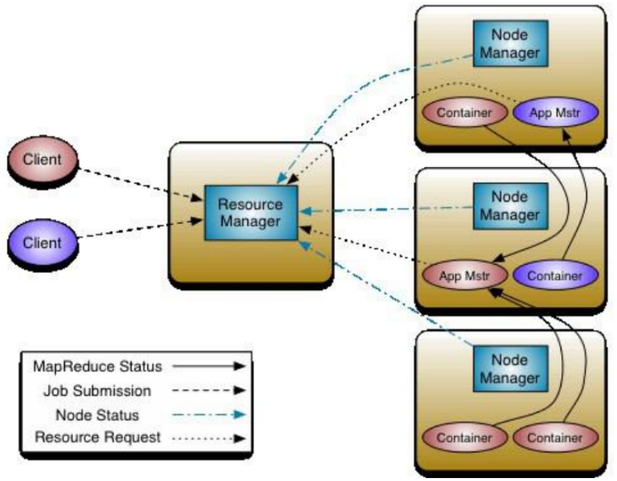
Master/Slave Architecture

主从式

- Key kernel functions always run on a particular processor (主核)
- Master is responsible for scheduling jobs
- Slave sends service request to the master
- Advantages:
  - is simple and requires little enhancement to a uniprocessor multiprogramming operating system
  - Conflict resolution is simplified because one processor has control of all memory and I/O resources
- Disadvantages:
  - failure of master brings down whole system
  - master can become a performance bottleneck

主x

性能瓶颈



Peer Architecture 对等式架构

- Kernel can execute on any processor
- Each processor does self-scheduling from the pool of available processes

Complicates the operating system

- operating system must ensure that two processors do not choose the same process and that the processes are not somehow lost from the queue

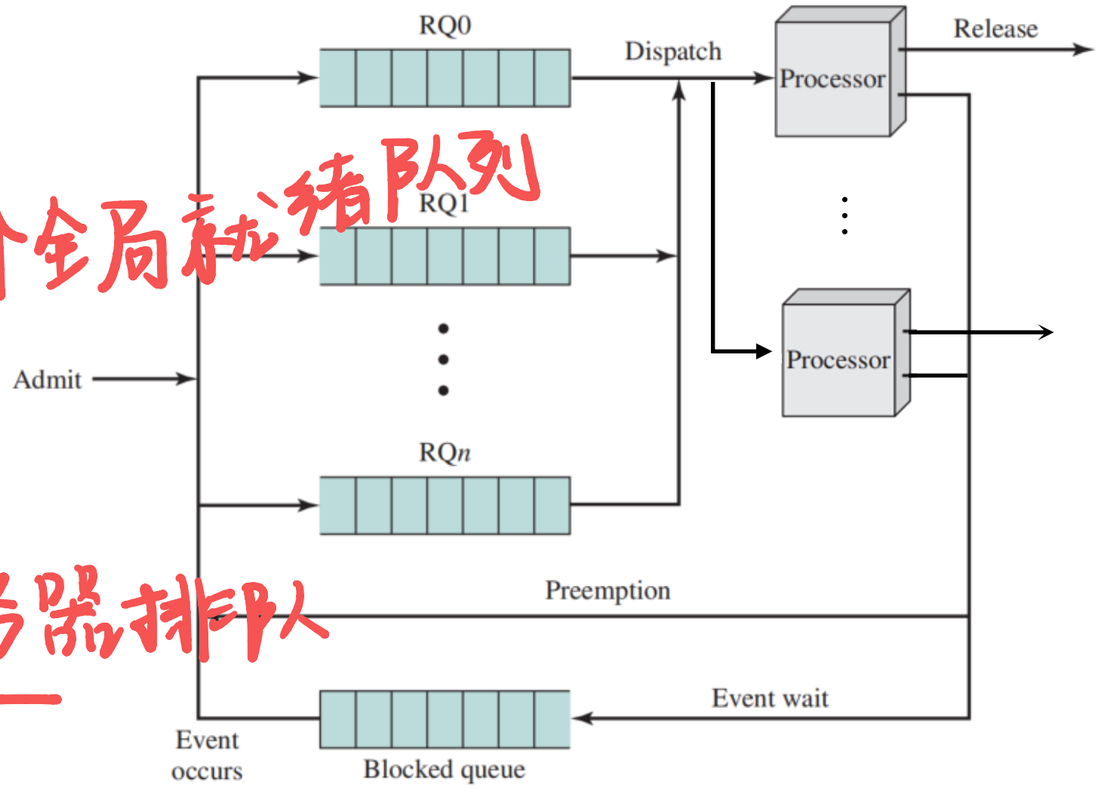
1.3 Process Scheduling in Multiprocessor System

进程不会被绑定到特定处理器（多核）

- Usually processes are not dedicated to a specific processor
- A single queue is used for all processors
  - if some sort of priority scheme is used, there are multiple queues based on priority
- System is viewed as being a multi-server queuing architecture

为所有处理器共用一个全局就绪队列

多服务器排队



Case of Dual-Processor System

- Comparison of scheduling performance

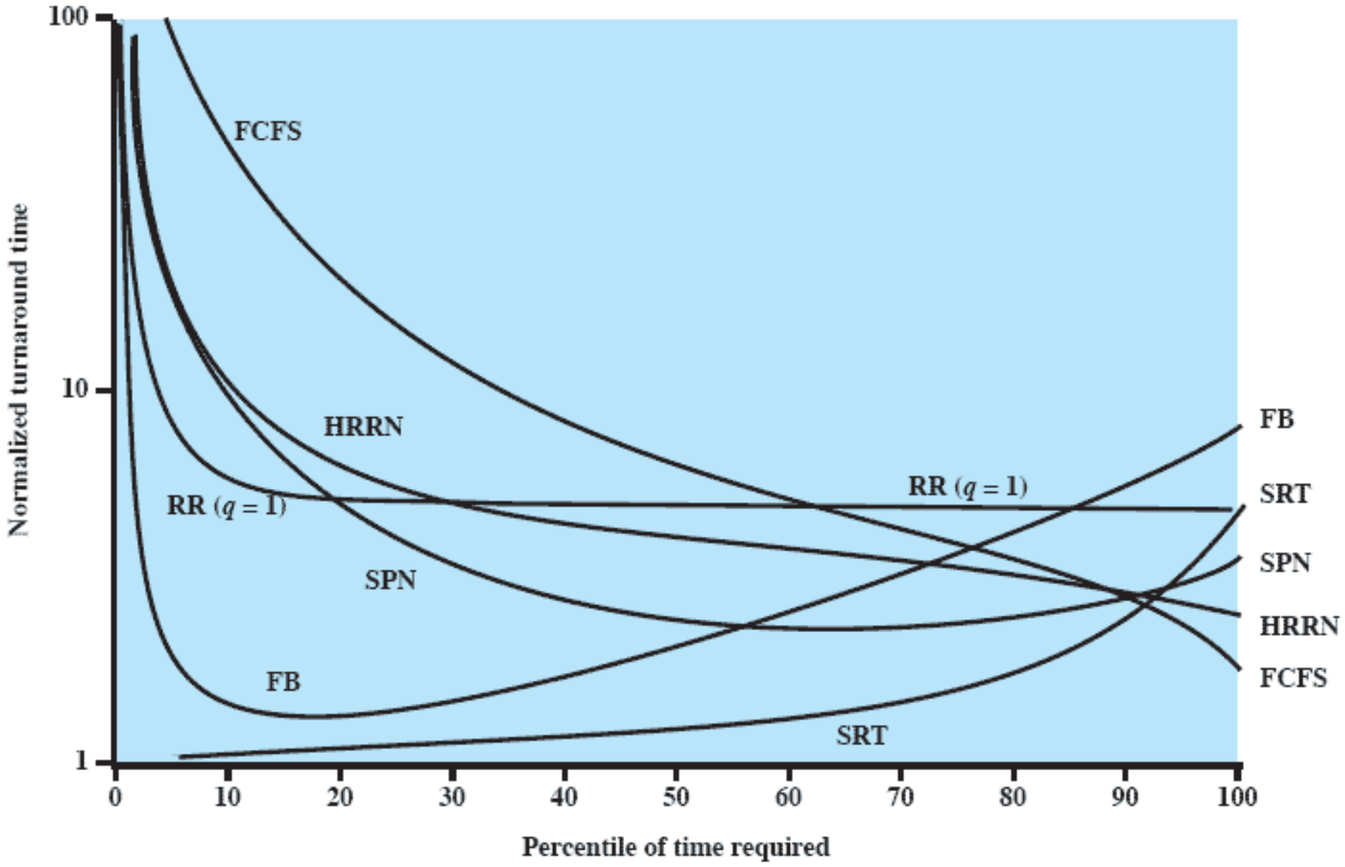
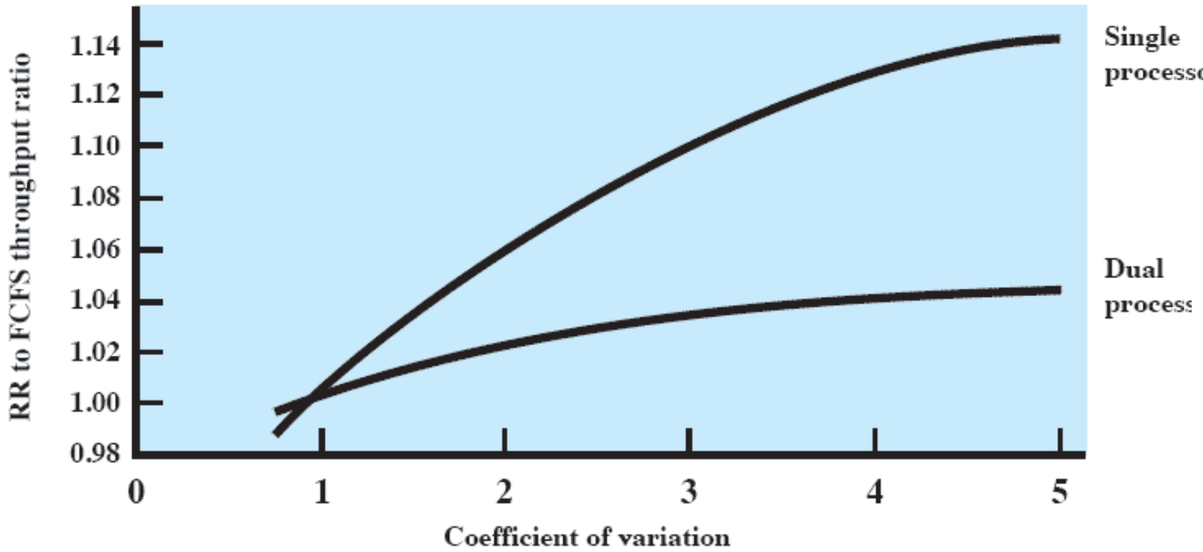


Figure 9.14 Simulation Results for Normalized Turnaround Time

Case of Dual-Processor System

- Comparison of scheduling performance

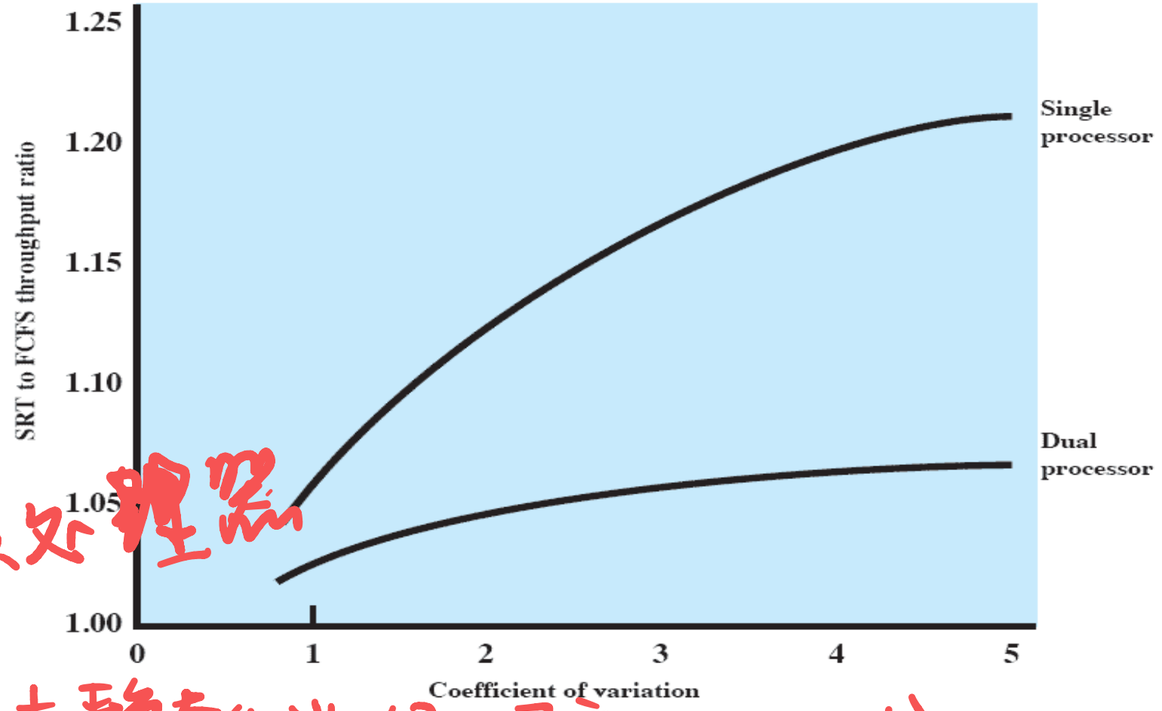
2核CPU



(a) Comparison of RR and FCFS

?

coefficient of variation –service time



(b) Comparison of SRT and FCFS

Conclusion： FCFS with a static priority scheme is suffice for MPS

↗多核处理器

多核环境下，FCFS+静态优先级调度=抢占式

1.4 Thread Scheduling in Multiprocessors

西南交通大学 Southwest Jiaotong University

- **Function: An application can be a set of threads that cooperate and execute concurrently in the same address space**
- **On a uniprocessor**
  - Threads can be used as a program structuring aid and to overlap I/O with processing (less overhead than process switch)
- **On a multiprocessor**
  - enable an application to simultaneously run on separate processors——Dramatic gains brought by multithreads in multiprocessor systems
  - require significant interaction among threads
  - Small differences in thread management and scheduling can have an impact on applications

Approaches to Thread Scheduling

西南交通大学 Southwest Jiaotong University

- **Four approaches for multiprocessor thread scheduling and processor assignment are:**
  - **Static Scheduling**
    - Load Sharing
      - processes are not assigned to a particular processor
        - When idle, each processor selects a thread from the global threads queue
    - Gang Scheduling
      - a set of related threads scheduled to run on a set of processors at the same time, on a one-to-one basis
    - Dedicated （专用） Processor Assignment
      - **Each program**, for the duration of its execution, is allocated **a number of processors** equal to the number of threads in the program
  - **Dynamic Scheduling**
    - the number of **threads in a process can be altered** during the course of execution

Load Sharing（全局就绪队列+自动拖任务）

西南交通大学 Southwest Jiaotong University

- Simplest approach and carries over most directly from a uniprocessor environment

Advantages:
• load is distributed evenly across the processors • no centralized scheduler required • the global queue can be organized and accessed using any of the schemes discussed in Chapter 9

自主调度

负载在所有处理器均匀分布

- Versions of load sharing: 变体
  - First-come-first-served (FCFS)
  - **Smallest number of threads first**: STF
    - ➤ The shared ready queue is organized as a priority queue, with **highest priority** given to threads from **jobs with the smallest number of unscheduled threads.**
    - ➤ a scheduled thread is run to **completion or blocking**.
  - **preemptive** smallest number of threads first

抢占式.

Disadvantages of Load Sharing

西南交通大学 Southwest Jiaotong University

- **Central queue occupies a region of memory**
  - **mutual exclusion access** among a large number of processors can lead to bottlenecks
- **Preemptive threads are likely to resume execution on the different processors**
  - **caching** can become less efficient
- **If all threads are treated as a common pool of threads, all of the threads of a program may not gain access to processors at the same time**
  - If **a high degree of coordination** is required between the threads of a program, the process switches involved may **seriously compromise** performance.

Gang Scheduling

成组调度

同时调度

线程

- Simultaneous scheduling of the threads that make up a single process
  - synchronization blocking may be reduced
  - less process switching may be necessary, and performance will increase
  - scheduling overhead may be reduced
- Useful for medium-grained to fine-grained parallel applications whose performance severely degrades when any part of the application is not running while other parts are ready to run
- Also beneficial for any parallel application

将构成单个进程的所有线程，同时调度

同步阻塞大量↓

调度开销↓

下降

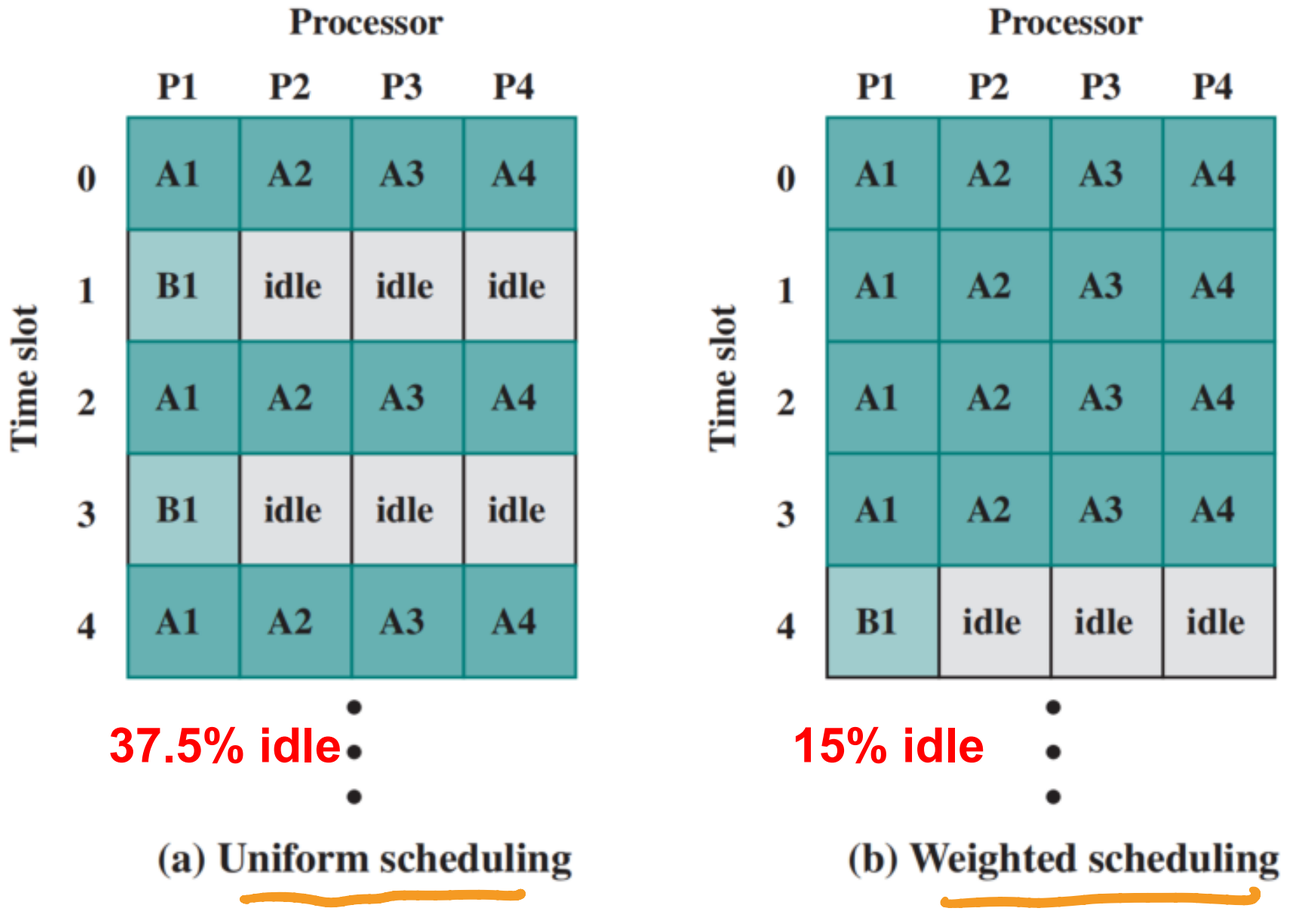
对所有并行应用性能↑

Example of Gang Scheduling

西南交通大学 Southwest Jiaotong University

- **The use of gang scheduling creates a requirement for processor allocation**

  - N processors
  - M applications,
  - Each application could be given **1/M** times



Example of scheduling groups with four and one threads

Dedicated Processor Assignment

专用处理器分配（0切换）

- When an application is scheduled, each of its threads is assigned to a processor that remains dedicated to that thread until the application runs to completion
- If a thread of an application is blocked waiting for I/O or for synchronization with another thread, then that thread's processor remains idle
  - there is no multiprogramming of processors
- Defense of this strategy: 合理性辩护
  - in a highly parallel system, with tens or hundreds of processors, processor utilization is no longer so important as a metric for effectiveness or performance
  - the total avoidance of process switching during the lifetime of a program should result in a substantial speedup of that program

避免切换

Application Speedup as a Function of Number of Threads

西南交通大学 Southwest Jiaotong University

Why it degrades after 8 threads ?

- **A greater frequency of thread preemption and rescheduling results in inefficiency from many sources**
  - waiting time for a suspended thread
  - Time wasted in process switching
  - inefficient cache behavior.

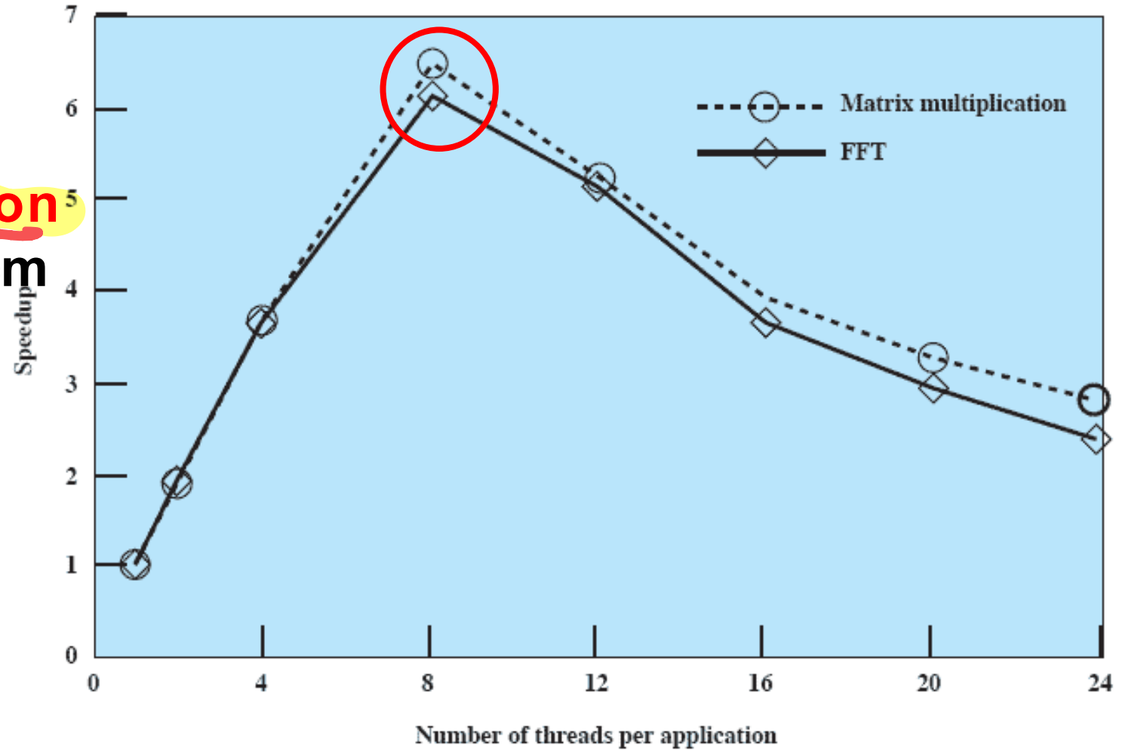


Figure 10.4 Application Speedup as a Function of Number of Threads

Dynamic Scheduling

西南交通大学 Southwest Jiaotong University

- **Some applications provide language and system tools to change the number of threads of process dynamically**
  - this would allow the operating system to adjust the load to improve utilization
- **Both the operating system and the application are involved in making scheduling decisions**
  - **The operating system**: is responsible for **partitioning the processors** among the jobs
  - **Individual application**: determine which subset to run, and **which thread to suspend** when a process is preempted
- **This approach is superior to gang scheduling or dedicated processor assignment for applications that can take advantage of it, BUT not for every application**

OS和APP共同决策

多个APP分配处理器资源

哪个子进程运行，哪个挂起

2 Real-Time Systems

西南交通大学 Southwest Jiaotong University

- **The operating system, and in particular the scheduler, is perhaps the most important component in Real-Time System**
  - control of laboratory experiments
  - process control in industrial plants
  - robotics
  - air traffic control
  - telecommunications
  - military command and control systems
- **Real-time computing: Correctness of the system depends on both the logical result of the computation and on the complete time.**
- **Real-time system:**
  - some of the tasks are **real-time tasks**, and these have a certain **degree of urgency** to them.
  - Such tasks are attempting to **control or react to events** that take place in the outside world.
  - Associate a **deadline** with a particular task

正确性=计算结果的逻辑+完成计算时间

实时任务、紧迫性

对外部的反映

Hard and Soft Real-Time Tasks

西南交通大学 Southwest Jiaotong University

- **Hard real-time task**
  - one that must **meet its deadline**
  - otherwise it will cause **unacceptable damage** or a **fatal error** to the system
- **Soft real-time task**
  - Has an associated deadline that is desirable but **not mandatory(不强制的)**
  - It still makes sense to schedule and complete the task even if it has passed its deadline

Another characteristic of real-time tasks

西南交通大学 Southwest Jiaotong University

- **Periodic tasks** 周期性任务.
  - requirement may be stated as:
    - once per **period T**
    - exactly **T units** apart 两次执行间隔时间严格为T个单位

非周期性

- **Aperiodic tasks**
  - has a deadline by which it must **finish or start**
  - may have a constraint on **both** start and finish time

约束

Characteristics of Real Time Systems

西南交通大学 Southwest Jiaotong University

- **Real-time operating systems have requirements in five general areas:**
  - Determinism
  - Responsiveness
  - User control
  - Reliability
  - Fail-soft operation（故障弱化）

Determinism

responsiveness

User control

reliability

fail-soft operation.

Determinism（可确定性）

西南交通大学 Southwest Jiaotong University

- **Determinism means that operations are performed at fixed, predetermined times or within predetermined time intervals**
  - when multiple processes are competing for resources and processor time, no system will be fully deterministic

预先确定的时间

- **Measure of determinism: the maximum delay from the arrival of a high-priority device interrupt to when servicing begins.**

- **The extent to which an operating system can deterministically satisfy requests depends on:**
  - the speed with which it can respond to interrupts
  - whether the system has sufficient capacity to handle all requests within the required time

中断响应速度

Responsiveness（可响应性）

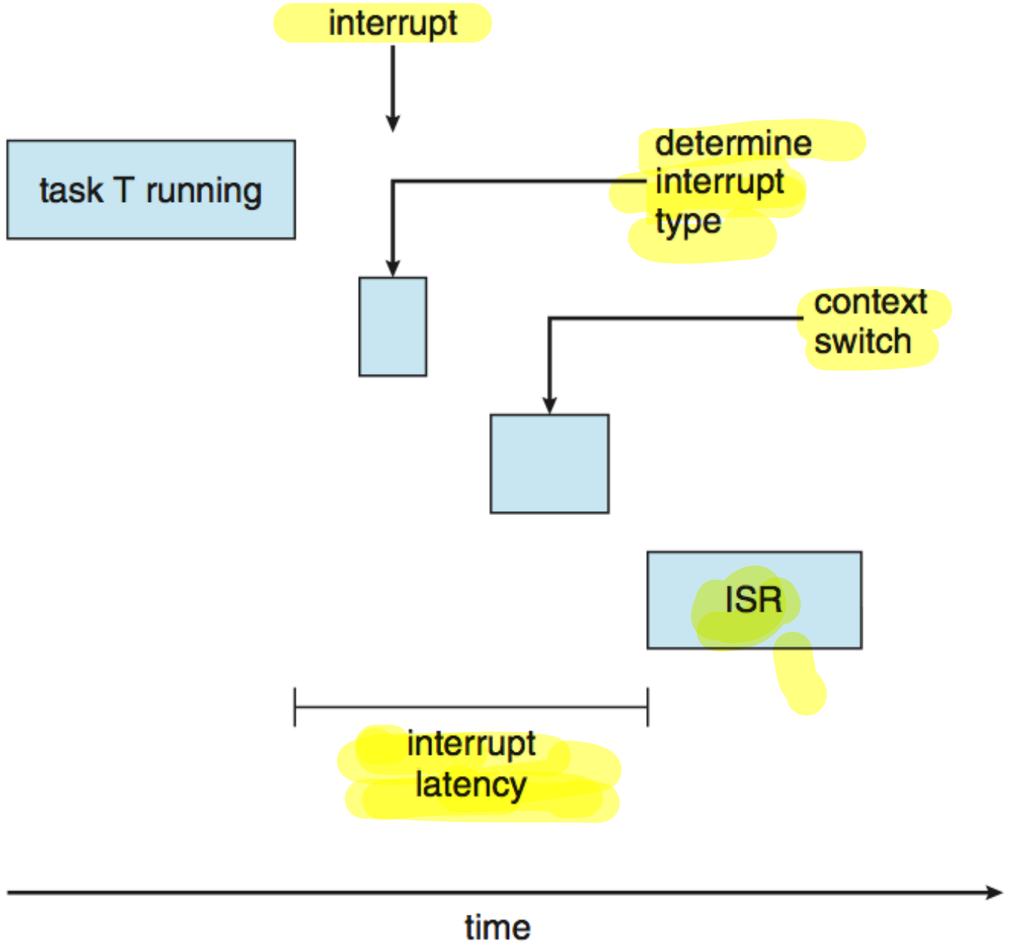
西南交通大学 Southwest Jiaotong University

- **Concerned with how long it takes an operating system to service the interrupt** 处理中断的用时
- **Responsiveness includes:**
  - amount of time required to initially handle the interrupt and begin execution of the interrupt service routine (ISR)
  - amount of time required to perform the ISR
  - effect of interrupt nesting(嵌套)

- **Together with determinism make up the response time to external events**
  - critical for real-time systems that must meet timing requirements imposed by individuals, devices, and data flows external to the system

对外部事件的响应时间 = ① determinism
② responsiveness

Interrupt


interrupt
task T running
determine
interrupt
type
context
switch
ISR
interrupt
latency
time

User Control

西南交通大学
Southwest Jiaotong University

用户控制权限更大

- Generally much broader in a real-time operating system than in ordinary operating systems
- It is essential to allow the user fine-grained control over task priority
- User should be able to distinguish between hard and soft real-time tasks and to specify relative priorities within each class
- May allow user to specify such characteristics as:
  - paging or process swapping
  - what processes must always be resident in main memory
  - what disk transfer algorithms are to be used
  - what rights the processes in various priority bands have

细粒度控制

任务优先级

Reliability（可靠性）

- **More important for real-time systems than non-real time systems**
- **Real-time systems respond to and control events in real time so loss or degradation of performance may have catastrophic consequences such as:**
  - financial loss
  - major equipment damage
  - loss of life

Fail-Soft Operation（故障弱化操作）提高容错率.

- A characteristic that refers to the ability of a system to fail in such a way as to preserve as much capability and data as possible
- Important aspect is stability

尽可能保留功能与数据

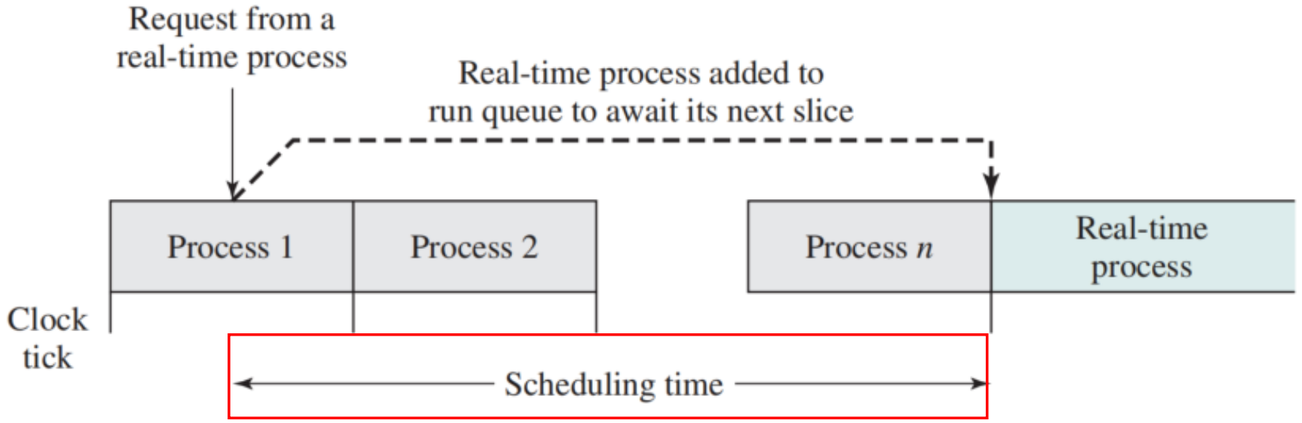
  - a real-time system is stable if the system will meet the deadlines of its most critical, highest-priority tasks even if some less critical task deadlines are not always met

Real-Time Scheduling of Process

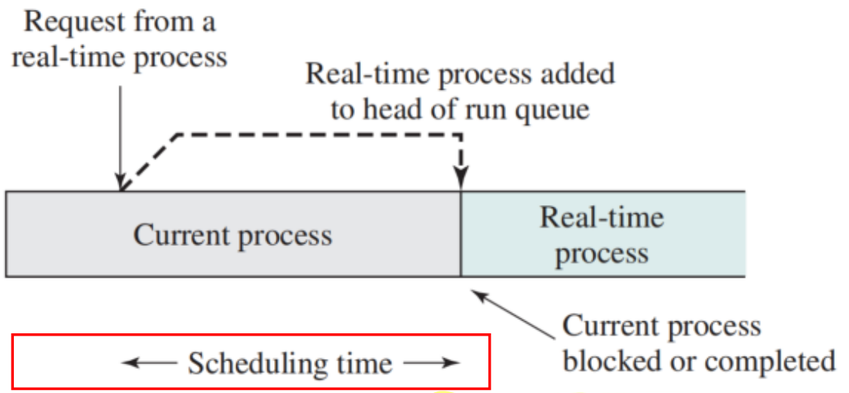
西南交通大学
Southwest Jiaotong University

- OS should be as responsive as possible to tasks with approaching deadlines
- when a deadline approaches, a task can be quickly scheduled

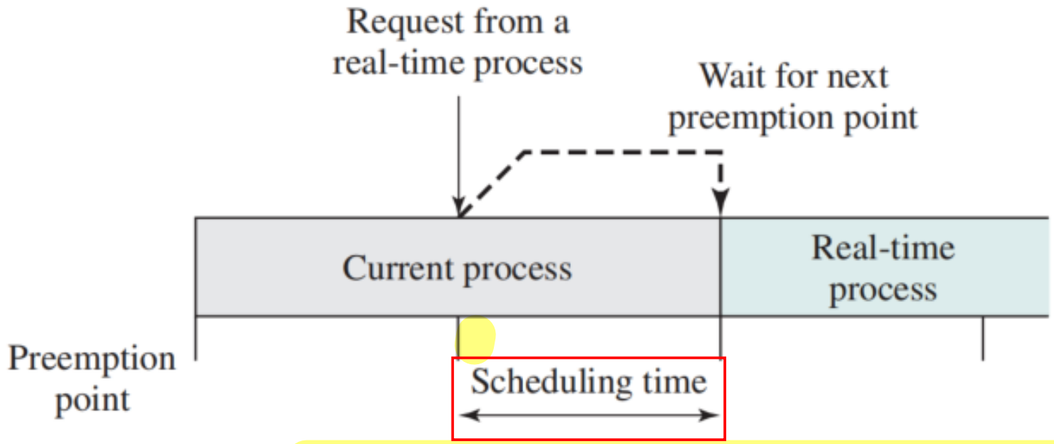
Scheduling of Real-Time Process



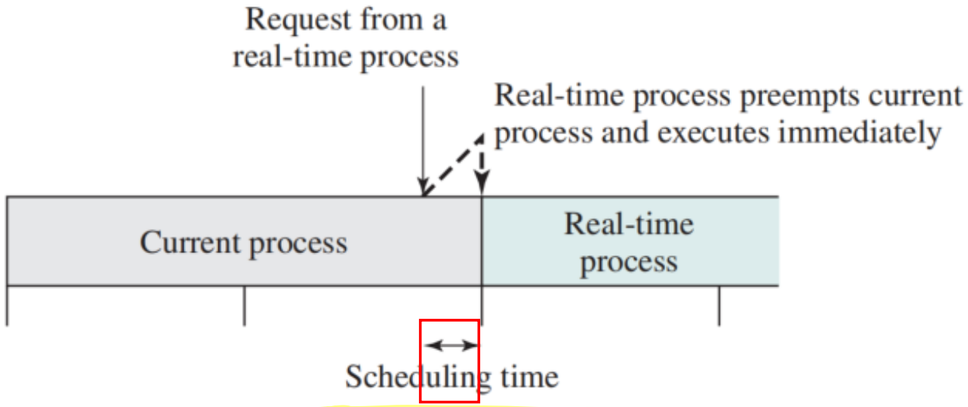
(a) Round-robin preemptive scheduler



(b) Priority-driven nonpreemptive scheduler



(c) Priority-driven preemptive scheduler on preemption points



(d) Immediate preemptive scheduler

Real-Time Scheduling

西南交通大学
Southwest Jiaotong University

(是否可 Deadline前)

- **Scheduling approaches depend on:**
  - whether a system performs schedulability analysis 是否执行可调度性分析
  - if it does, whether it is done statically or dynamically
  - whether the result of the analysis itself produces a scheduler plan according to which tasks are dispatched at run time

Classes of Real-Time Scheduling Algorithms

西南交通大学 Southwest Jiaotong University

- **Static table-driven approaches (periodic)**
  - performs a static analysis of feasible schedules of dispatching
  - result is a schedule that determines when a task must begin execution
- **Static priority-driven preemptive approaches**
  - a static analysis is performed but no schedule is drawn up
  - analysis is used to assign priorities to tasks so that a traditional priority-driven preemptive scheduler can be used
- **Dynamic planning-based approaches** 动态规划
  - feasibility is determined at run time rather than offline prior to the start of execution
  - one result of the analysis is a schedule that is used to decide when to dispatch this task
- **Dynamic best effort approaches**
  - no feasibility analysis is performed
  - system tries to meet all deadlines and aborts any started process whose deadline is missed

中止任何已开始但错过截止时间的进程

Deadline Scheduling

- Real-time operating systems are designed with the objective of starting real-time tasks as rapidly as possible

- OS emphasizes rapid interrupt handling and task dispatching

不追求绝对速度

- Real-time applications are generally not concerned with sheer speed but rather with completing tasks at the most valuable times

而是追求在最关键时间完成任务

- Priorities provide a crude tool and do not capture the requirement of completion at the most valuable time

优先级→粗糙的工具

Information Used for Deadline Scheduling

西南交通大学 Southwest Jiaotong University

- **Ready time**
  - time task becomes ready for execution
- **Starting deadline**
  - time task must begin
- **Completion deadline**
  - time task must be completed
- **Processing time** (处理时间)
  - time required to execute the task to completion
- **Resource requirements**
  - resources required by the task while it is executing
- **Priority**
  - measures relative importance of the task
- **Subtask structure** 子任务结构 强制子任务 可选子任务
  - a task may be decomposed into a mandatory subtask and an optional subtask
  - Only the mandatory subtask possesses a hard deadline.

强制子任务 有硬截止时间

Critic Design Issues of Real-Time Scheduling

西南交通大学 Southwest Jiaotong University

- **When deadline is considered:**
  - Which task to schedule next, and what sort of preemption is allowed.

- **For a given preemption strategy**
  - Using starting or completion deadlines to decide which task

- **The other critical design issue is that of preemption.**
  - When starting deadlines are specified, then a nonpreemptive scheduler makes sense. (need to block itself)
  - For a system with completion deadlines, a preemptive strategy is most appropriate.

Table 10.2 Execution Profile of Two Periodic Tasks

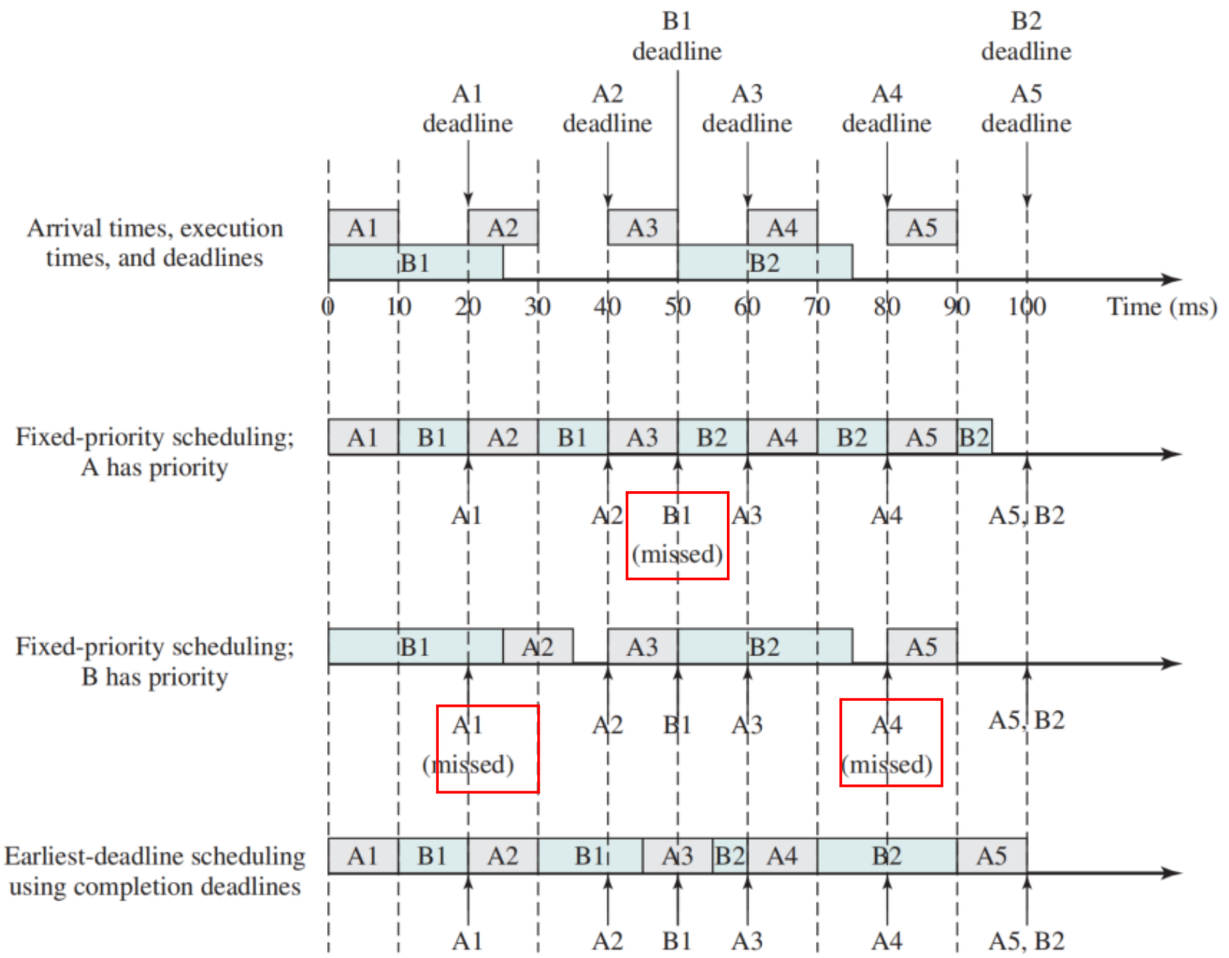
西南交通大学 Southwest Jiaotong University

T.
A: 20ms.
B: 50ms

Process	Arrival Time	Execution Time	Ending Deadline
A(1)	0	10	20
A(2)	20	10	40
A(3)	40	10	60
A(4)	60	10	80
A(5)	80	10	100
•	•	•	•
•	•	•	•
•	•	•	•
B(1)	0	25	50
B(2)	50	25	100
•	•	•	•
•	•	•	•
•	•	•	•

Scheduling of Periodic Real-Time Tasks with Completion Deadlines

西南交通大学
Southwest Jiaotong University



Because the tasks are periodic and predictable, a static table-driven scheduling approach is used

静态表驱动调度方法

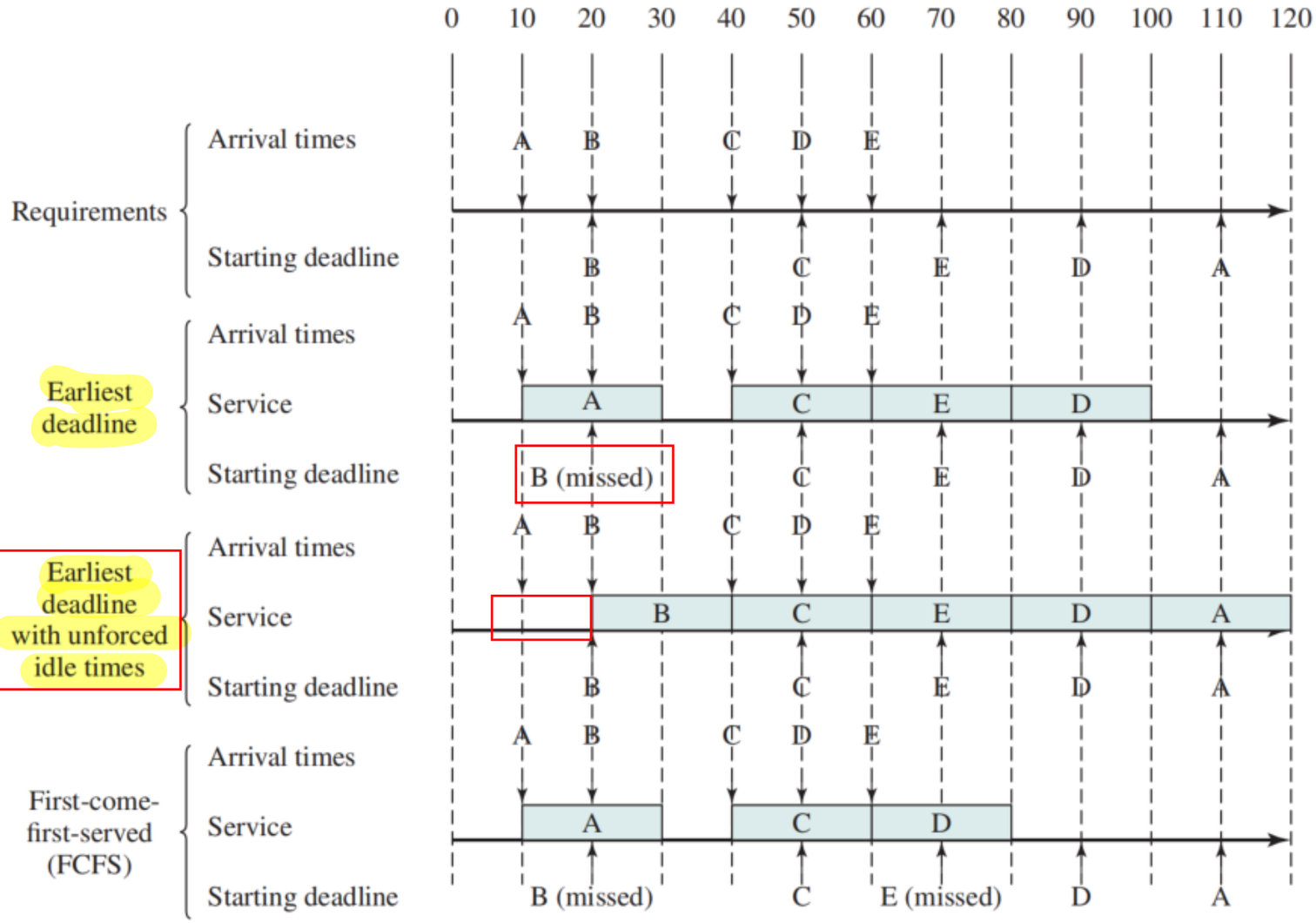
Table 10.3 Execution Profile of Five Aperiodic Tasks

西南交通大學 Southwest Jiaotong University

Process	Arrival Time	Execution Time	Starting Deadline
A	10	20	110
B	20	20	20
C	40	20	50
D	50	20	90
E	60	20	70

Scheduling of Aperiodic Real-Time Tasks with Starting Deadlines

- Non-Preemption



Rate Monotonic Scheduling 速度单调调度

- the highest-priority task is the one with the shortest period
- T: Task's Period (second)
- Rate = 1/T (Hz)
  - T=50ms
  - Rate = 1/0.05=20 Hz
- Priority is monotonically increasing with respect to rate

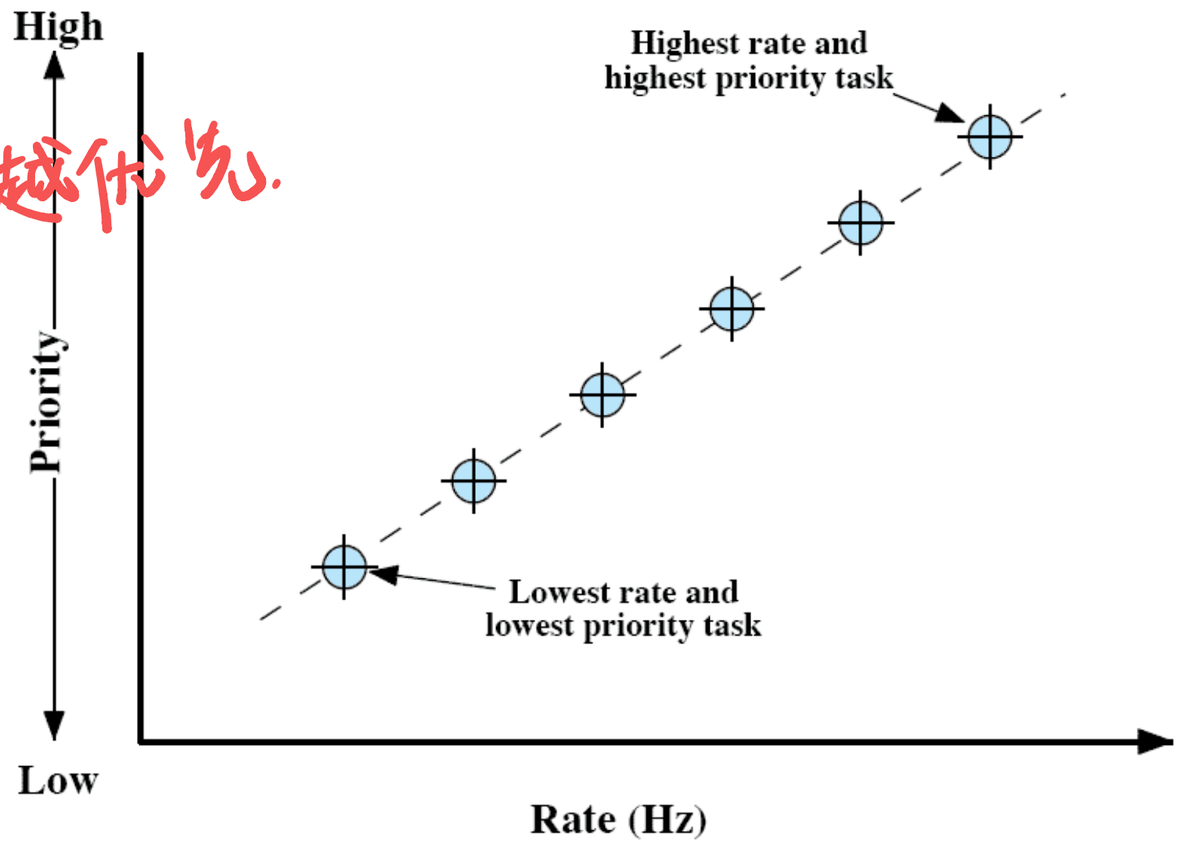
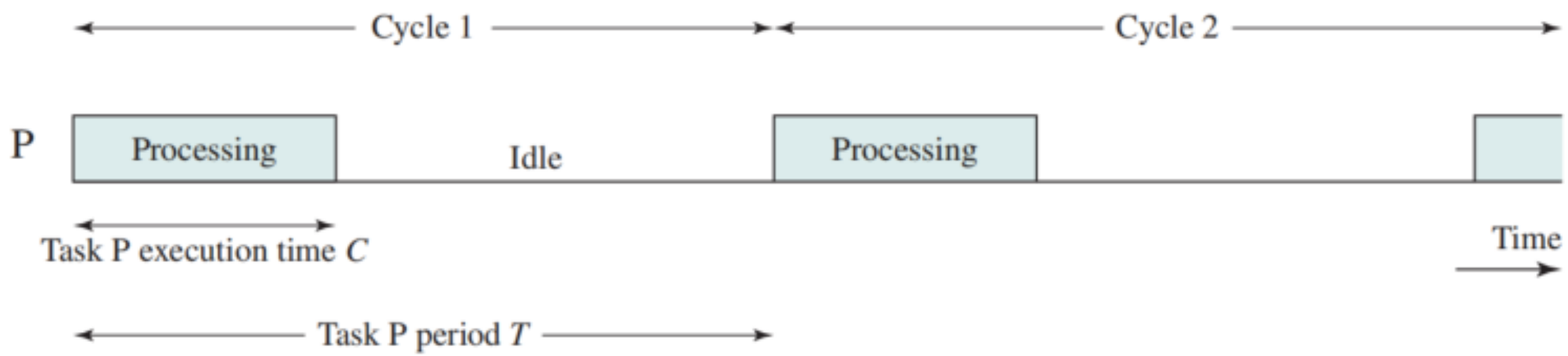


Figure 10.8 A Task Set with RMS [WARR91]

Periodic Task Timing Diagram

- C: execution time
- T: period
- U = C/T: processor utilization



Measure of Periodic Scheduling Algorithm

- If a **perfect** scheduling algorithm can successfully schedule, a bound of number of tasks is

$$\frac{C_1}{T_1} + \frac{C_2}{T_2} + \ \ldots \ \ + \frac{C_n}{T_n} \leq 1$$

- Any particular algorithm, bound may be lower:

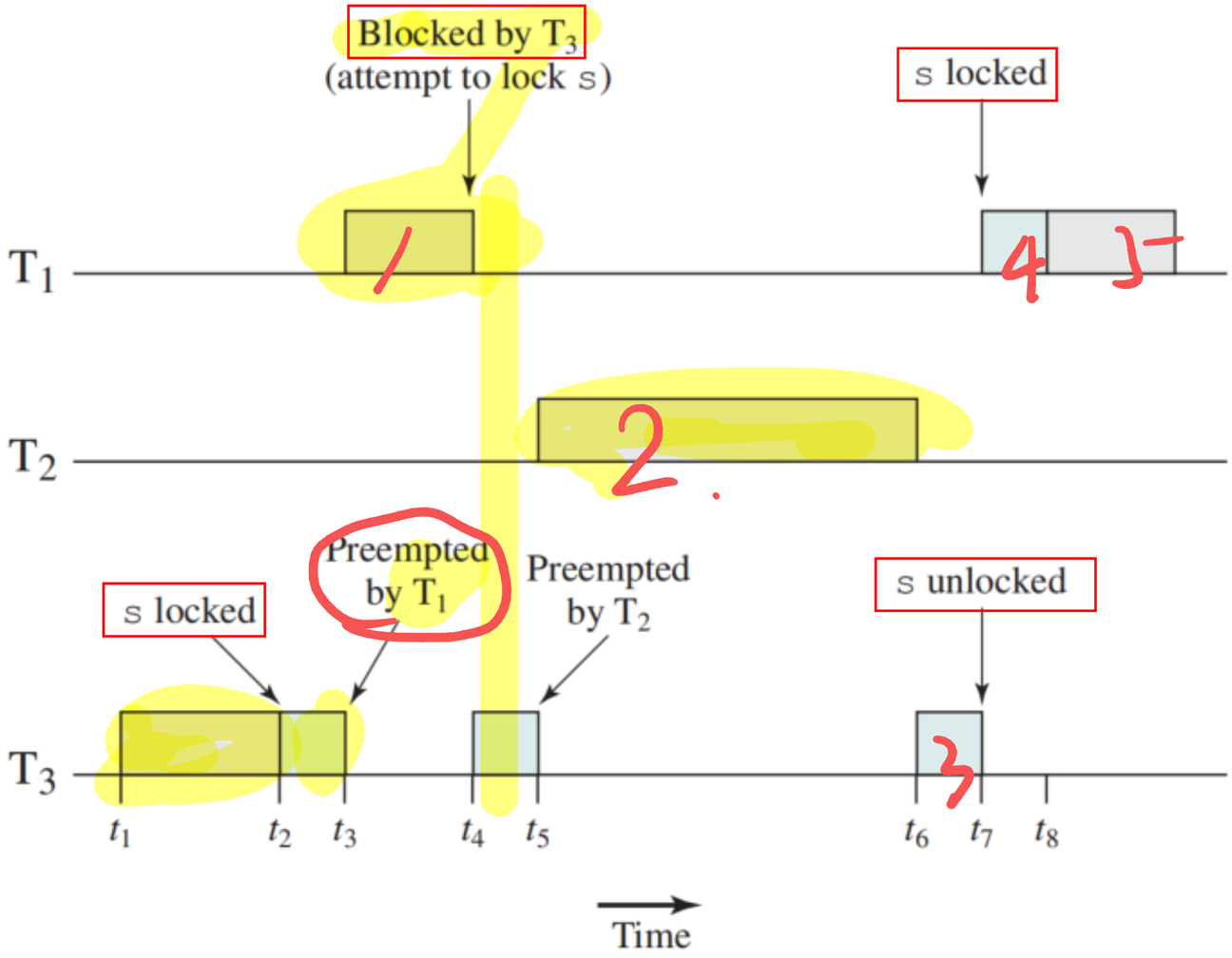
$$\frac{C_1}{T_1} + \frac{C_2}{T_2} + \ \ldots \ \ + \frac{C_n}{T_n} \leq n(2^{1/n} - 1)$$

n	$n(2^{1/n}-1)$
1	1.0
2	0.828
3	0.779
4	0.756
5	0.743
6	0.734
•	•
•	•
•	•
∞	$\ln 2 \approx 0.693$

Priority Inversion 优先级反转

- **Occurs when circumstances within the system force a higher priority task to wait for a lower priority task**
  - Can occur in any priority-based preemptive scheduling scheme
  - Particularly relevant in the context of real-time scheduling
- **Best-known instance involved the Mars Pathfinder mission**
- **Unbounded Priority Inversion**
  - the duration of a priority inversion depends not only on the time required to handle **a shared resource**, but also on the **unpredictable actions of other unrelated tasks**

Unbounded Priority Inversion



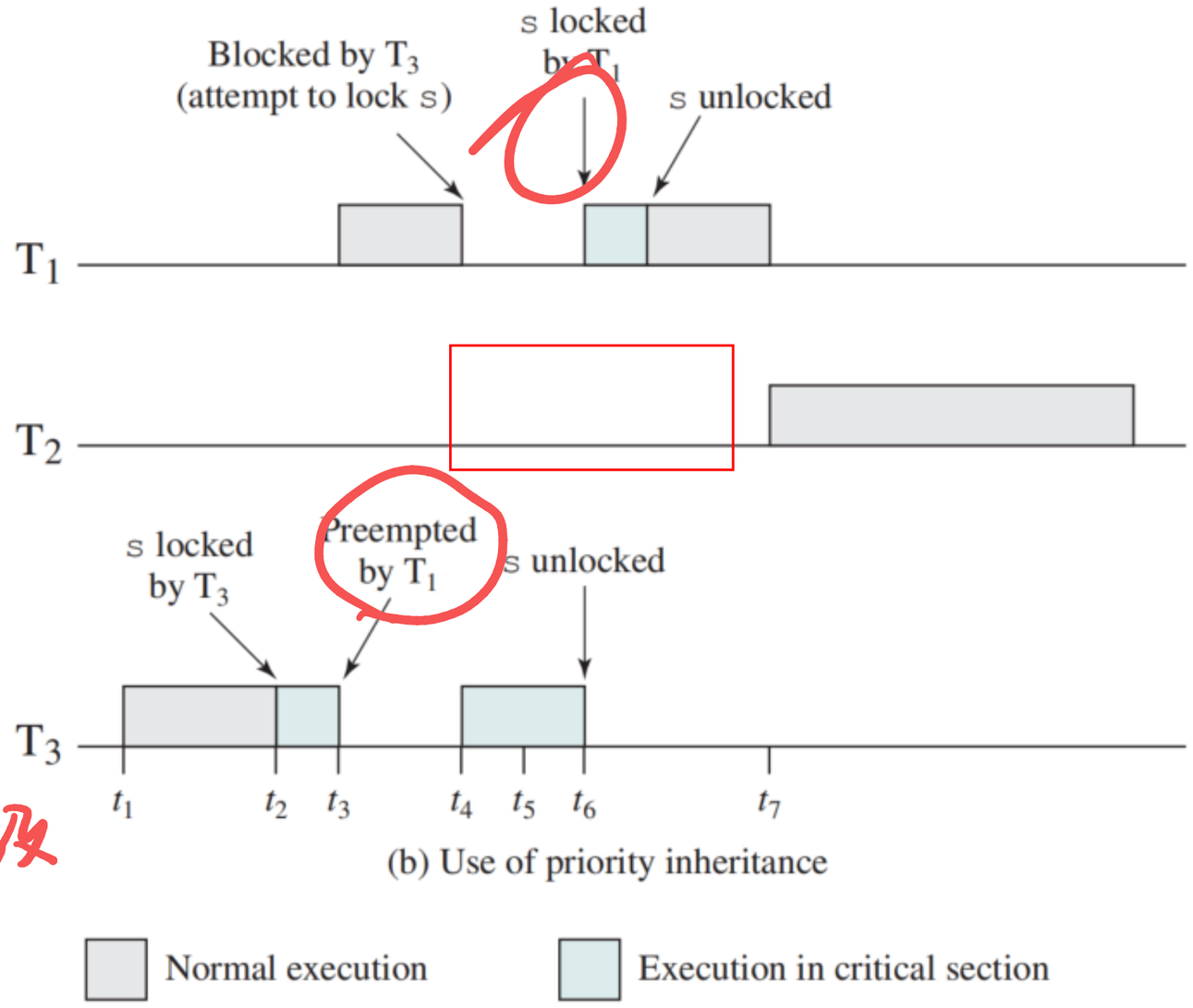
(a) Unbounded priority inversion

Priority T1>T2>T3

Priority Inheritance

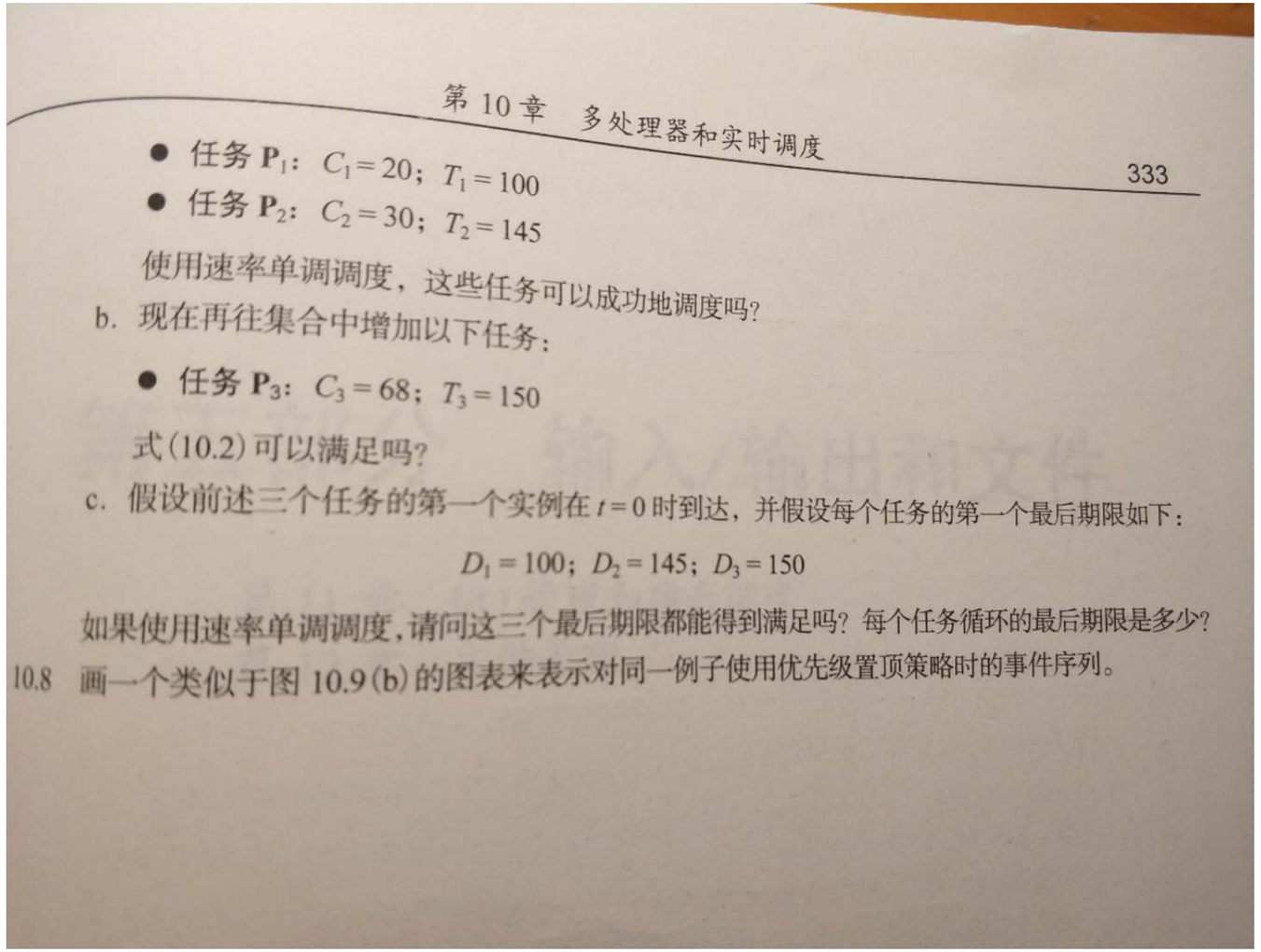
- a lower-priority task inherits the priority of any higher-priority task pending on a **resource they share**.

低优先级任务继承所有等待共享资源的高优先级任务



Quiz

- 对于速率单调调度，有如下独立的周期任务



- 任务 $\mathbf{P}_1$：$C_1 = 20$；$T_1 = 100$
- 任务 $\mathbf{P}_2$：$C_2 = 30$；$T_2 = 145$

使用速率单调调度，这些任务可以成功地调度吗？

b. 现在再往集合中增加以下任务：

- 任务 $\mathbf{P}_3$：$C_3 = 68$；$T_3 = 150$

式(10.2)可以满足吗？

c. 假设前述三个任务的第一个实例在 $t = 0$ 时到达，并假设每个任务的第一个最后期限如下：

$$D_1 = 100;\ D_2 = 145;\ D_3 = 150$$

如果使用速率单调调度，请问这三个最后期限都能得到满足吗？每个任务循环的最后期限是多少？

10.8　画一个类似于图 10.9(b)的图表来表示对同一例子使用优先级置顶策略时的事件序列。

Summary

- With a tightly coupled multiprocessor, multiple processors have access to the same main memory
- Performance studies suggest that the differences among various scheduling algorithms are less significant in a multiprocessor system
- A real-time process is one that is executed in connection with some process or function or set of events external to the computer system and that must meet one or more deadlines to interact effectively and correctly with the external environment
- Algorithms that rely heavily on preemption and on reacting to relative deadlines are appropriate in this context